Pollinator Habitat — Developer Handoff

Last updated: 2026-04-19

This document is the starting point for any developer taking over this project. It covers current state, known gotchas, and the fastest path to getting productive. Read this first, then follow the links into the deep-dive docs.

What the Project Is

A **nature walk game** for visitors at Conner Prairie. Players scan a QR code and are automatically connected to the active session on their phone, then follow a guided route through the park — stopping at marked locations to learn facts about pollinators. There is no app store install; it runs entirely in the mobile browser.

A separate **staff portal** lets Conner Prairie staff search party-size survey data collected during sessions and export results to CSV.

Three apps, one backend, one database:

```
Frontend (Next.js) ——— player phones
Portal   (Next.js) ——— staff laptops
         ↓ both talk to ↓
Backend  (Express)
         ↓
Database (MySQL or PostgreSQL — your choice)
```

The full architecture is in OVERVIEW.md.

Current State of Each App

Frontend

All pages implemented and working:

Page	Route
Auto-connect splash	`/`
Home	`/home`
Route walk	`/route`
Pollinator collection	`/pollinator-collection`
Accessibility settings	`/accessibility`
Redirect	`/redirecting`

Accessibility settings: High Contrast Mode, Text-to-Speech toggle, Font Size Slider (75/100/125/150%). All settings persist to `localStorage["accessibilitySettings"]` and are applied via CSS classes or the `--font-scale` CSS custom property.

Backend

34 API endpoints fully implemented:

- Session validation and player session management
- Route assignment with deduplication/cycle logic
- Survey response recording
- Pollinator name lookup
- Route Statistics — 20 statistical query endpoints across 10 metrics, both session-scoped and date-range-scoped

JWT authentication guards all game endpoints. The portal's statistical queries are unauthenticated (internal-only design decision).

Full endpoint list: API_SPECIFICATION.md

Portal

Feature	Status
Session ID search (survey data)	✅
Date-range search (survey data)	✅
Survey CSV export (from survey page)	✅
Quick Export Tool — Survey Report (by period)	✅
Quick Export Tool — Route Report (by period)	✅
Route stats browser (session-level, route/page.tsx)	🚧 Placeholder — "Coming Soon"

Known Gotchas

MySQL and PostgreSQL Are Both Supported

The project ships two complete database stacks — MySQL and PostgreSQL. Both work for development and production. Prisma uses a driver-adapter pattern so the same schema and codebase targets either engine.

Choose based on your deployment constraints. If MySQL's GPL license is a concern for your use case, use PostgreSQL. Details: MYSQL_LICENSE_ISSUE.md

---

Font Scale Does Not Persist Across Cold Starts on Other Pages

The `--font-scale` CSS property (and the `high-color-contrast` / `TTS-option-switch` classes) are only written to `document.documentElement` by `accessibility/page.tsx`. In a Next.js client-side navigation these persist, but on a **hard refresh or new tab** they reset to defaults. The user must visit the accessibility page again to re-apply their preferences.

This is a known limitation, not a bug.

Pre-Seeded Session IDs

The seed creates one session per calendar day from **2023-04-17 through 2026-04-16**. Session IDs follow the pattern `YYYYMMDD0` (9 digits), e.g.:

Date	Session ID
2023-04-17	`202304170`
2024-04-17	`202404170`
2025-04-17	`202504170`
2026-04-16	`202604160`

These are useful when searching the portal — enter a 9-digit session ID into the session ID search field, or use the date-range search to query across any date window in that range. The game frontend does not ask for a session ID; it auto-connects to whichever session the server currently has active.

Prisma Requires OpenSSL at Build Time

The backend Dockerfile explicitly installs OpenSSL in the build stage. Without it, Prisma's query engine binary fails to generate. If you see a Prisma binary error during a fresh Docker build, check that the OpenSSL install step is present in backend/Dockerfile.pg.

Compose Files

There are four compose files — two database engines × two environments. Migrations and seed data run automatically in all four via a one-shot `migrate` container that completes before the backend starts.

File	Environment	Database	When to use
`docker-compose.dev.yml`	Development	MySQL 8.4	Local dev with MySQL; hot-reload, source volumes mounted
`docker-compose.dev.pg.yml`	Development	PostgreSQL 18	Local dev with PostgreSQL; hot-reload, source volumes mounted
`docker-compose.yml`	Production	MySQL 8.4	Production deploy with MySQL; nginx reverse proxy, no exposed app ports
`docker-compose.prod.pg.yml`	Production	PostgreSQL 18	Production deploy with PostgreSQL; nginx reverse proxy, no exposed app ports

Container inventory (same across all four stacks)

Container	Role
`backend`	Express API — port 4000 (dev) or internal-only via nginx (prod)
`frontend`	Next.js player app — host port 80 (dev) or internal via nginx (prod)
`portal`	Next.js staff portal — host port 3001 (dev) or internal via nginx (prod)

`mysql` / `postgres`	Database — host port 3306/5432 (dev) or internal-only (prod)
`migrate`	One-shot container: runs migrations + seed, then exits
`nginx`	Reverse proxy — **prod only**; routes port 80 → frontend, port 3001 → portal

Env files

Both env files are committed to the repo with working defaults:

- **`.env`** — used by the MySQL stacks (`docker-compose.yml` , `docker-compose.dev.yml`). Contains `MYSQL_*` credentials, `DATABASE_URL` , `SHADOW_DATABASE_URL` , and `JWT_SECRET` . Docker Compose reads it automatically.
- **`.env.pg.example`** — used by the PostgreSQL stacks (`docker-compose.prod.pg.yml` via `env_file:` , and `docker-compose.dev.pg.yml` via `${JWT_SECRET}` which Docker Compose resolves from `.env.pg.example` if present). Contains `POSTGRES_*` credentials, `DATABASE_URL` , and `JWT_SECRET` .

> ***Before production deployment:*** *replace* `JWT_SECRET` *in both files with a fresh random value.*

Full variable list: ENV_VARIABLES.md

Architecture Decisions (Quick Reference)

Decision	What was chosen	Why
Database	MySQL or PostgreSQL (both supported)	Prisma adapter pattern keeps a single schema; choose engine per deployment
ORM	Prisma with driver adapter	Decouples engine choice from application code
Auth	JWT (frontend only)	Portal is internal — no auth needed there
Accessibility persistence	`localStorage` only	No backend dependency; applied entirely client-side
CSS units	`rem` everywhere	Enables font scaling via `--font-scale` on `<html>`
Route data	Database only	`Routes.json` was removed; source of truth is the seed file → DB

How to Get Running (5 Minutes)

Both `.env` (MySQL) and `.env.pg.example` (PostgreSQL) are already committed with working default credentials — no setup needed.

With PostgreSQL:

```
cd Pollinator-Habitat/
docker compose -f docker-compose.dev.pg.yml up --build
```

With MySQL:

```
cd Pollinator-Habitat/
docker compose -f docker-compose.dev.yml up --build
```

Migrations and seed run automatically on first start. No manual seed step needed.

Frontend: http://localhost
Portal: http://localhost:3001
Backend: http://localhost:4000

Full setup with troubleshooting: DOCKER_SETUP.md
All environment variables: ENV_VARIABLES.md

Running Tests

```
cd Pollinator-Habitat/

npm run test:backend    # all backend unit + integration tests
npm run test:frontend   # all frontend tests
npm run test:portal     # all portal tests
```

All 37 test files documented: TEST_DOCUMENTATION.md

Key Docs Map

Question	Where to look
What does each API endpoint do?	API_SPECIFICATION.md
How does the route-walk logic work?	NEXT_ROUTE_FLOW.md
What's in the database?	DATABASE_SCHEMA.md
How do I add a new pollinator route?	ADDING_ROUTES.md
How does the portal work?	PORTAL_FLOW.md
How does accessibility work?	ACCESSIBILITY.md
What do all the tests do?	TEST_DOCUMENTATION.md
How do I deploy this?	Admin/ADMIN_GUIDE.md
What are the Route Statistics endpoints?	ROUTE_STATISTICS_SPEC.md